Visual Relocation Markers

1. Course Content

Course Overview

In robot autonomous navigation, **LiDAR (Light Detection and Ranging) localization** is extremely precise, but in the following typical industrial scenarios, it is prone to **"localization degradation and drift"**:

1. **Long straight corridors and symmetric shelf passages**: Both sides are completely featureless columns with identical characteristics - LiDAR "looks the same everywhere". When the chassis wheels experience slight slipping, the cart has difficulty determining whether it moved forward 1 meter or 1.2 meters;
2. **Human traffic and dynamic obstacle occlusion**: When LiDAR's field of view is largely blocked by surrounding obstacles, scan matching scores drop sharply;
3. **Long-term operation cumulative error**: After running for tens of minutes, tiny wheel speed integral errors may accumulate to 5-10 cm, causing the robotic arm to miss objects during grasping operations.

The cart introduces an industrial-grade multi-sensor fusion architecture of **"LiDAR localization as primary + visual tag codes (AprilTag/ArUco) global repositioning redundant calibration"**.

Learning Objectives

1. Deeply understand the geometric calculation principle of AprilTag / ArUco tags in robot visual repositioning.
2. Master the process of graphically checking in and recording tag code global coordinates using `apriltag_global_recorder.launch.py` tool.
3. Master the data structure of `aritag_relocation.yaml` configuration file and on-site tag size parameter tuning.

---

2. Preparation

2.1 Map and Route Network Preparation

Please ensure you have completed the sandbox environment grid map and pose graph construction according to the previous lesson [Build Sand Table Map], and planned the basic driving route network.

2.2 AprilTag Tag Size Confirmation and Parameter Configuration

- The standard sandbox map tag codes included in the kit (usually 36h11 tag family), have a factory-calibrated actual physical black frame outer edge length of **0.048 meters (4.8 cm)**:
- Core parameter configuration file path:

```
~/yahboomcar_ros2_ws/dofbot_pro_ws/src/dofbot_pro_bringup/config/common.yaml
```

- Confirm the actual physical size `tag_size` of the tag in the parameter file:

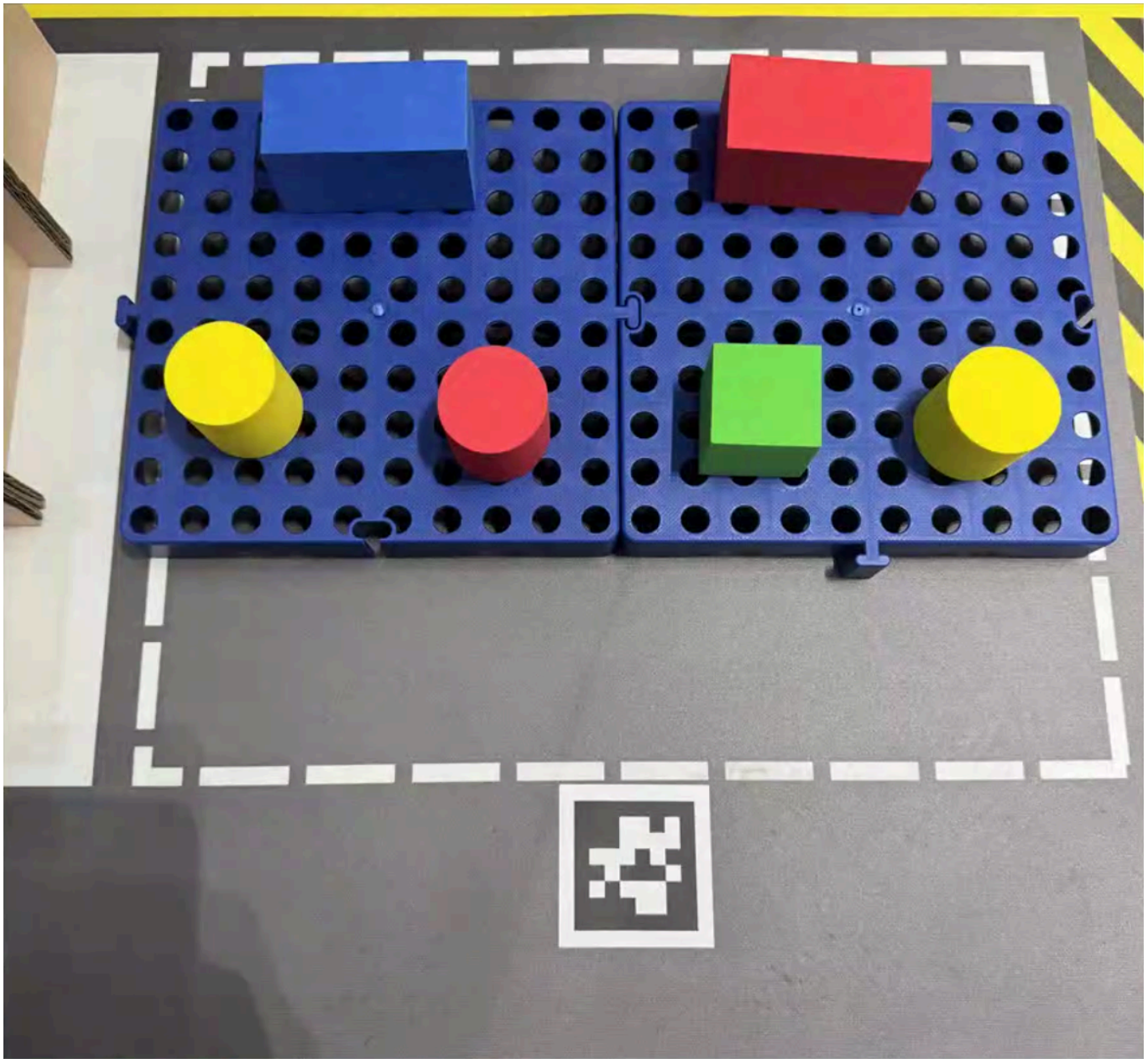
```
...
apriltag:                # Tag detection node configuration
  ros__parameters:
    image_transport: raw  # Image format
    family: 36h11         # Tag family (default is tag36h11)
    size: 0.048           # Tag black outer frame actual physical edge length, unit: meters
...
```

> **Important**
> If you print your own tag paper, please measure the black frame outer edge length with a ruler and accurately fill in the real meter value (for example, 5cm fill `0.050`) into `size`, otherwise spatial ranging inverse solution will have proportional scaling deviation.

---

3. Deploy ArUco Tag Codes in Sandbox Map

- **Deployment Principle**: Tag codes should be pasted at positions where the **onboard camera can clearly see the front or at a large angle** when the robot navigates into a workstation (for example, shelf front columns, operation table side panels, or wall edges, recommended height 15-25 cm from ground).
- Recommended sandbox tag deployment reference positions are as follows:





> **Important**
>
> - The visual repositioning mechanism is automatically triggered when the cart approaches the target workstation. If the tag code is seen in the camera's field of view upon arrival, the system immediately starts global repositioning to accurately position the cart; if the tag code is not seen due to angle issues, the system automatically falls back to using laser odometry for `PreciseAdjustPosition` fine-tuning, the two are mutually redundant.

---

3. Preparation and Execution (Phase 1: Record Tag Code Global Positions)

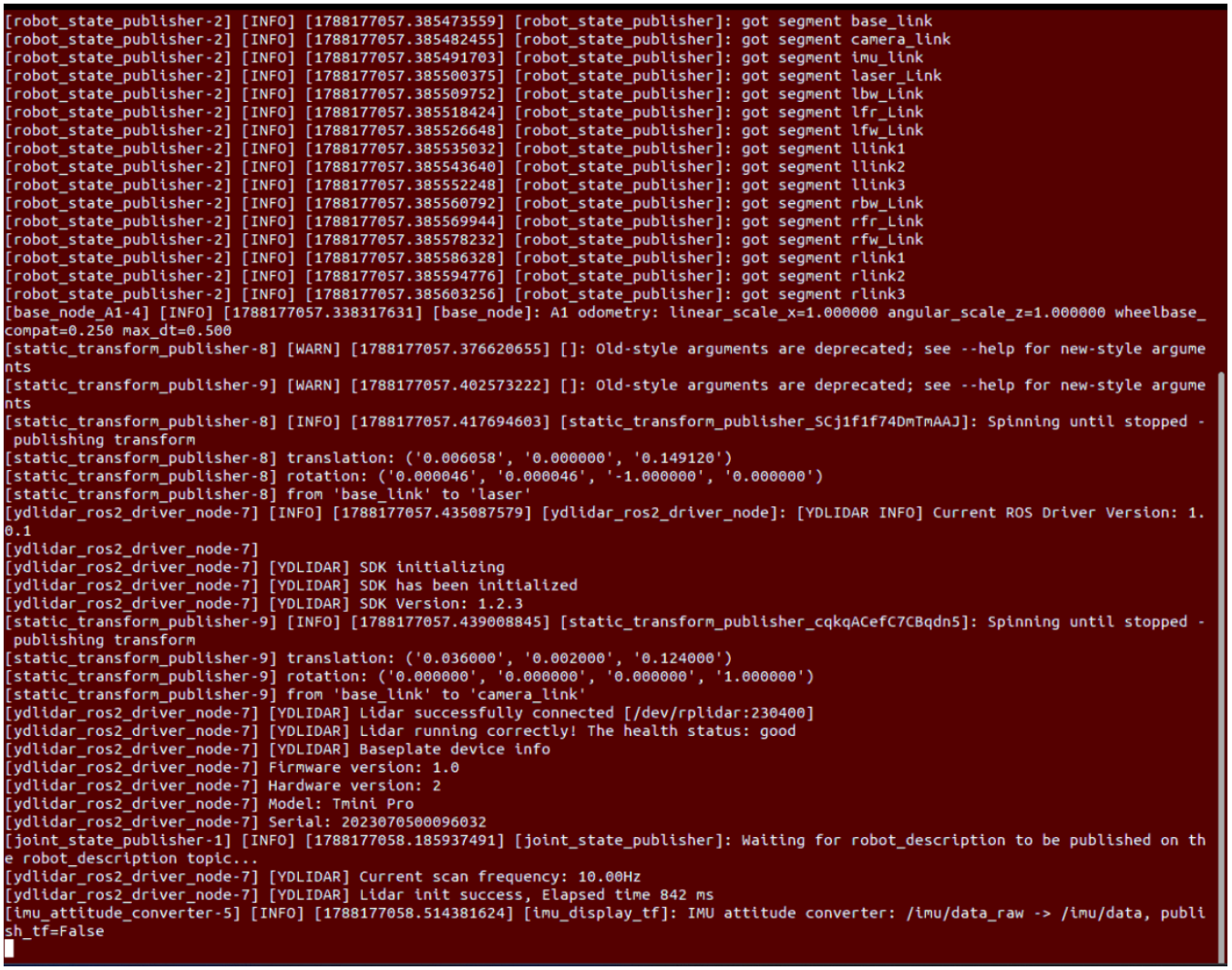
Before formally using visual repositioning, you need to record and save the global coordinates of the tag codes arranged on the wall or ground in a high-precision LiDAR localization environment.

This phase requires starting the following nodes in sequence:

3.1 Start Chassis and Sensors

Run in **Terminal 1** (start chassis drive, LiDAR, and RGB depth camera):

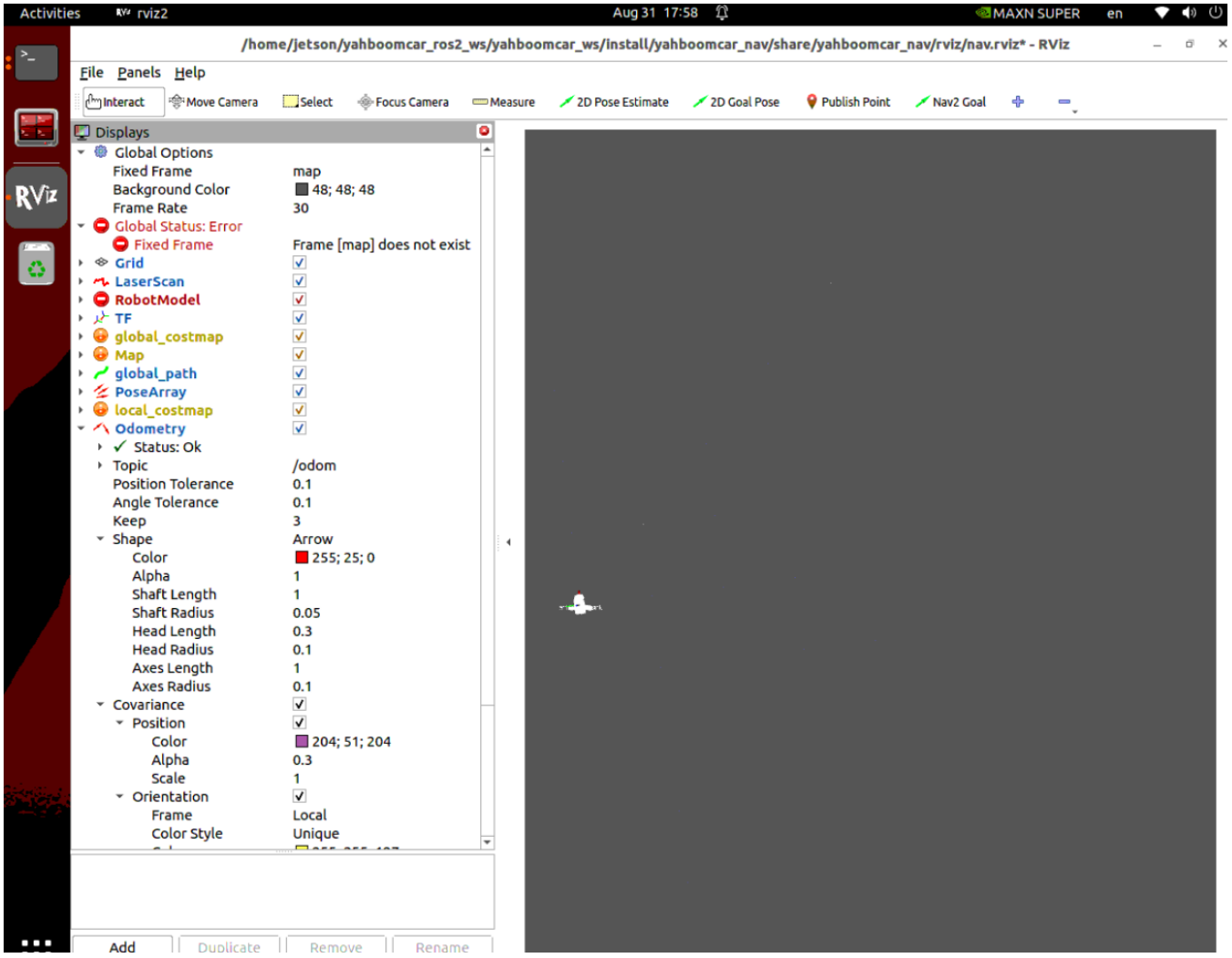
```
ros2 launch yahboomcar_nav laser_bringup_launch.py
```



3.2 Start RViz Navigation Visualization

Run in **Terminal 2** (recommended to open RViz on the virtual machine side when virtual machine and cart are on the same LAN):

```
ros2 launch yahboomcar_nav display_nav_launch.py
```



3.3 Start Cartographer Pure Localization Node

Load `.pbstream` pose graph in **Terminal 3** to start pure localization core:

```
ros2 launch yahboomcar_nav localization_imu_odom.launch.py use_rviz:=false
load_state_filename:=$HOME/map/map.pbstream
```

3.4 Start Nav2 Navigation Controller

Start navigation controller in **Terminal 4** (load corresponding static grid map):

```
ros2 launch yahboomcar_nav navigation_cartodwb_launch.py maps:=$HOME/map/map.yaml
```

> **Tip**
>
> After pure localization starts, if in a low-feature area, you can use the `2D Pose Estimate` tool in the RViz top toolbar to assist in aligning the initial pose:



3.5 Start AprilTag Recording Mode

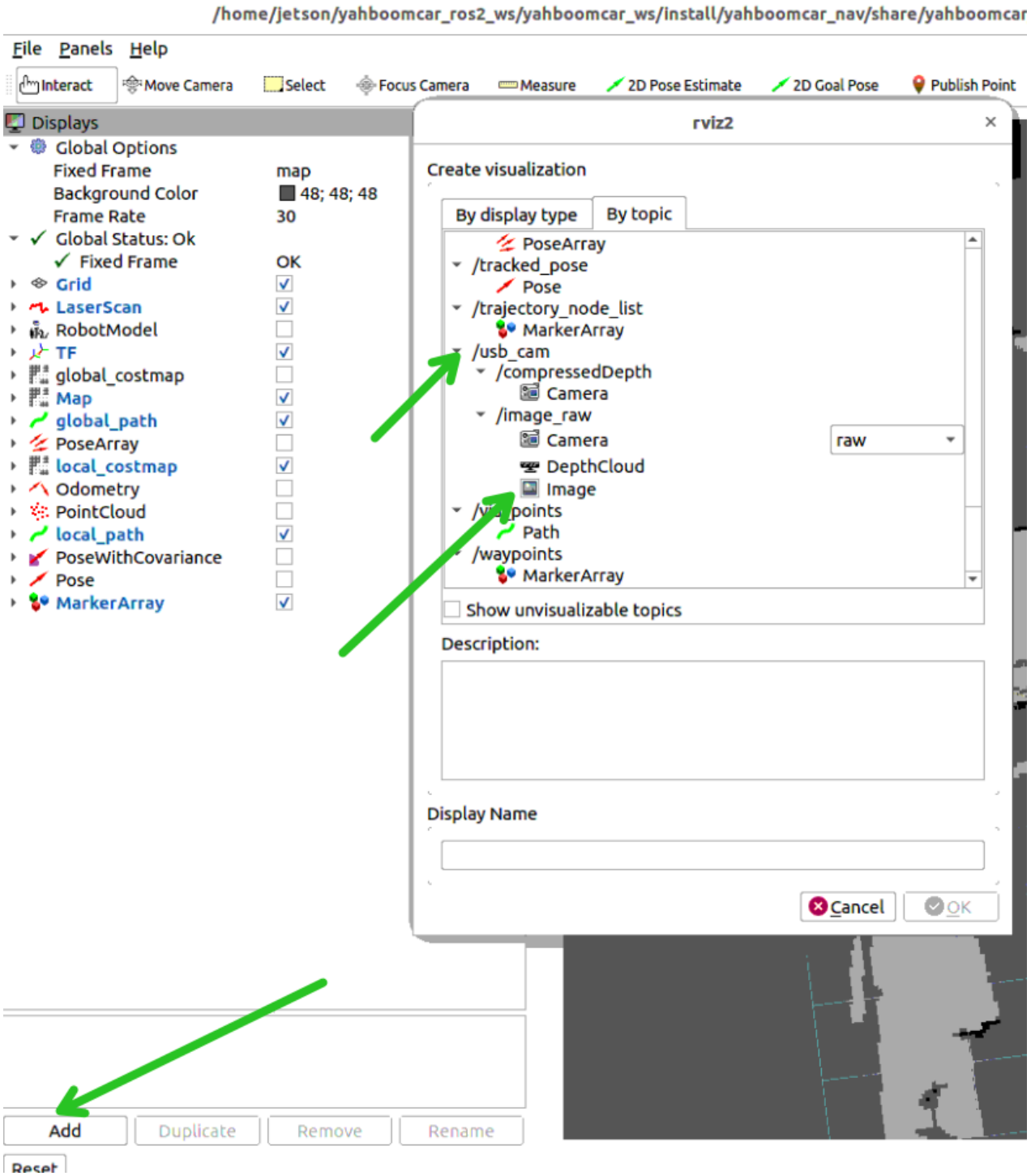
Run in **Terminal 5** (start detection node and pose recorder):

```
ros2 launch yahboomcar_nav apriltag_relocation.launch.py mode:=record
```

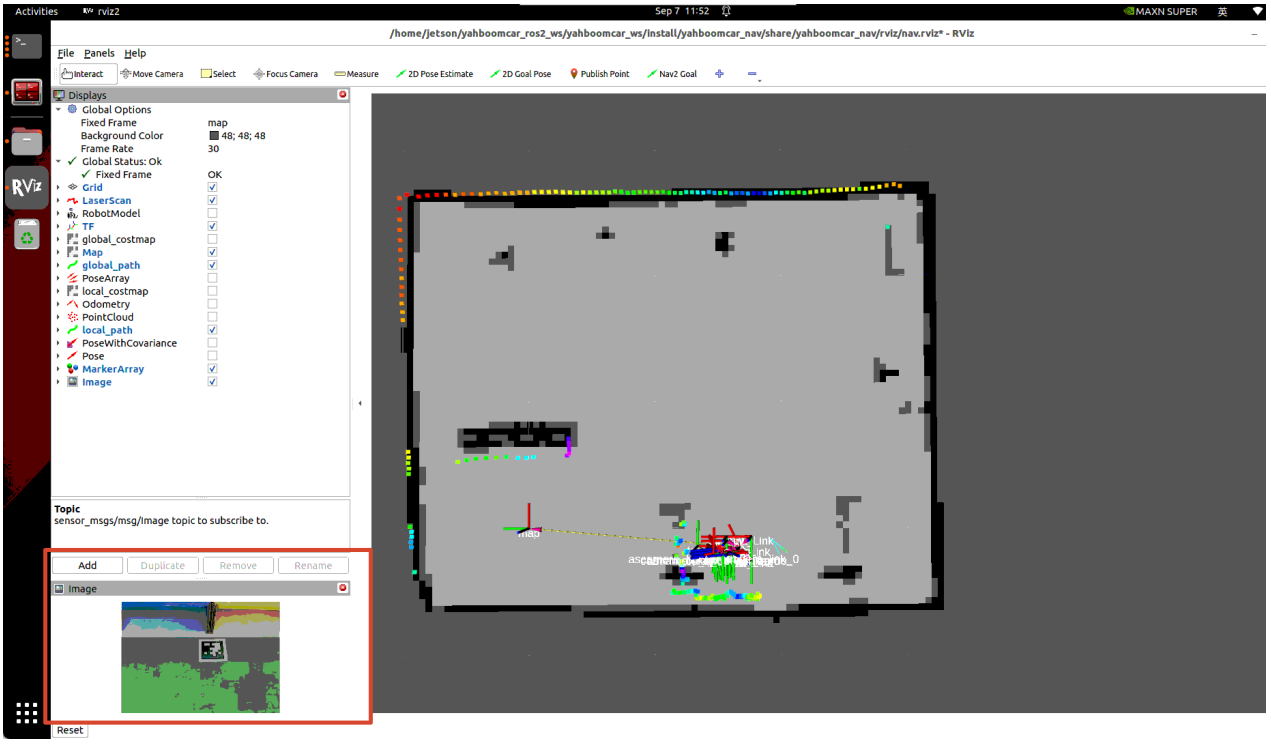
```
jetson@yahboom:~/yahboomcar_ros2_ws/yahboomcar_ws$ ros2 launch yahboomcar_nav apriltag_relocation.launch.py mode:=record
[INFO] [launch]: All log files can be found below /home/jetson/.ros/log/2026-08-31-21-00-51-195326-yahboom-225659
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [apriltag_node-1]: process started with pid [225734]
[INFO] [apriltag_recorder-2]: process started with pid [225736]
[apriltag_recorder-2] [INFO] [1788181251.453186133] [apriltag_recorder]: AprilTag Recorder initialized
[apriltag_recorder-2] [INFO] [1788181251.453375100] [apriltag_recorder]: Output file: /home/jetson/yahboomcar_ros2_ws/dofbot_pro_ws/insta
l/dofbot_pro_bringup/share/dofbot_pro_bringup/config/aritag_relocation.yaml
[apriltag_recorder-2] [INFO] [1788181251.453399292] [apriltag_recorder]: Target frame: map
[apriltag_recorder-2] [INFO] [1788181251.459118402] [apriltag_recorder]: Service 'save_april_tag_pose' is ready
[apriltag_node-1] [INFO] [1788181251.839750346] [apriltag]: AprilTagNode initialized. Detection enabled: true
```

1. Click `Add` in the bottom-left corner of RViz2 interface, select `By topic` $\rightarrow$

`/apriltag/image_rect` to add Image plugin for real-time camera detection view:



2. Remotely control the cart to drive to the front of the tag code, ensure the camera image clearly shows the tag code with 3D coordinate axes and green detection box:



3.6 View Tag ID and Record Pose

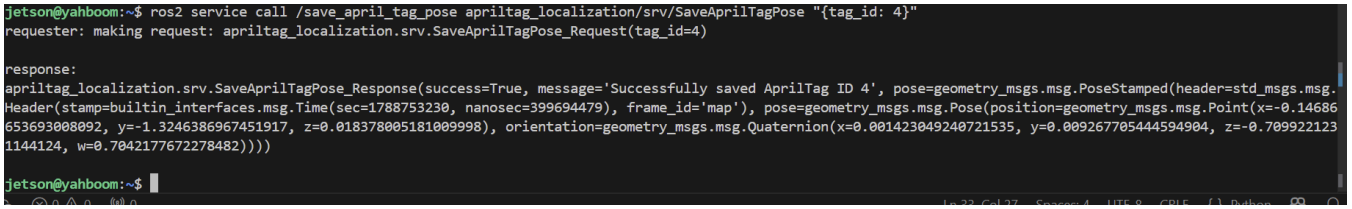
View currently detected tag ID in **Terminal 6**:

```
ros2 topic echo /detections --once
```

```
jetson@yahboom:~$ ros2 topic echo /detections --once
header:
  stamp:
    sec: 1788753203
    nanosec: 779727055
  frame_id: ascamera_hp60c_color_0
detections:
- family: tag36h11
  id: 4
  hamming: 0
  goodness: 0.0
  decision_margin: 122.35743713378906
  centre:
    x: 368.1943992566147
    y: 206.3950264907012
  corners:
  - x: 323.5022888183594
    y: 169.9187164306641
  - x: 324.10754394531244
    y: 246.1982421875
  - x: 416.6273193359375
    y: 245.9244689941406
  - x: 409.0436401367188
    y: 169.5148468017578
  homography:
  - -12.372080688124916
  - -44.06918350927458
  - 368.1943992566147
  - 30.03464454850092
  - 0.3839276623813587
```

After confirming the detected ID (for example, ID is 4), call the service to persistently write the tag's global coordinates in the map to the configuration file:

```
ros2 service call /save_april_tag_pose apriltag_localization/srv/SaveAprilTagPose "{tag_id: 4}"
```



> **Note**
>
> After successful recording, the system writes the tag's precise pose in the `map` coordinate system to `aritag_relocation.yaml`. If there are multiple tag codes, remotely control the cart to each tag in sequence and call the above service.